

Syntaxprüfer für While- und Loop-Programme

Lehrstuhl für Theoretische Informatik, Universität Würzburg

22. Oktober 2010

1 Installation

Zum Betrieb des Syntaxprüfers ist Python erforderlich. Abhängig vom Betriebssystem kann Python über die jeweilige Paketverwaltung oder manuell installiert werden. Die offizielle Python-Webseite lautet <http://python.org/>. Neben der genauen Sprachdefinition und Referenz zur Standardbibliothek ist dort unter anderem die jeweils aktuelle Python-Version zu finden.

Der Syntaxprüfer benötigt mindestens Python 2.6. Es wird jedoch empfohlen, mindestens Python 3.0 zu installieren, da sich die späteren Erweiterungen von While- und Loop-Programmen auf Sprachfeatures dieser Python-Version beziehen können.¹

2 Bedienung

Der Syntaxprüfer wird auf der Kommandozeile benutzt. Dazu ist dem Aufruf von Python das Verzeichnis, in das der Syntaxprüfer entpackt wurde, als Parameter mitzugeben. Alle dann nachfolgenden Parameter betreffen den Syntaxprüfer selbst, wobei die erlaubten Parameter im Folgenden erläutert werden. Ein möglicher Aufruf des Syntaxprüfers zusammen mit der resultierenden Ausgabe wäre zum Beispiel:

```
goll@thomson:~$ python syntax-checker -e fac.txt 7
[+] <WHILE> fac.txt is a while program.

[-] <LOOP> fac.txt is not a loop program because of the following:
      Calling function 'fac' from within its own definition not allowed. (line 12, column 15)

      1.12:      res = mul(fac((n - 1)), n)
                  ^

I will now try and evaluate the function defined by fac.txt.

---[ start of print output ]-----
1
2
6
24
120
720
5040
---[ end of print output ]-----

The result is:
main(7) = 5040
```

Hier weist die an den Syntaxprüfer übergebene Option „-e“ den Syntaxprüfer an, das in „**fac.txt**“ abgelegte Programm nicht nur auf While- und Loop-Konformität zu prüfen, sondern auch die durch das Programm definierte, im Beispiel 1-stellige Funktion an der Stelle 7 auszuwerten und das Ergebnis auszugeben.

Ruft man den Syntaxprüfer mit der Option „-h“ auf, so wird eine kurze Auflistung der erwarteten Parameter sowie der definierten Optionen ausgegeben. Die erlaubten Optionen werden im Folgenden erläutert.

¹Die Unterschiede zwischen Python 2.x und Python 3.x sind zum Teil beträchtlich.

2.1 Allgemeine Optionen

- version** Gibt die aktuelle Programmversion des Syntaxprüfers aus. Bei der Meldung von Fehlern im Syntaxprüfer ist es zur Reproduktion und Behebung des Fehlers nötig, die verwendete Version des Programms zu identifizieren. Dies kann mit dieser Option erreicht werden.
- h, --help** Zeigt die Kurzübersicht zur Bedienung des Syntaxprüfers an.
- v, --verbose** Erhöht den Detailgrad der während des Programmlaufs ausgegebenen Meldungen; die Option kann mehrmals angegeben werden, um weitere Meldungen anzuzeigen. Im Normalfall gibt der Syntaxprüfer nur das Ergebnis „erfüllt While- bzw. Loop-Spezifikation“ mit etwaigen Fehlerstellen aus. Durch das Aktivieren von „--verbose“ werden weitere Status- und Fortschrittsmeldungen sowie interne Datenstrukturen ausgegeben.

2.2 Optionen zum Verhalten

- e, --evaluate** Weist den Syntaxprüfer an, die durch das While- bzw. Loop-Programm definierte Funktion auszuwerten. Die Argumente der Funktion werden dabei nach dem Dateinamen angegeben, mit einem Parameter je Funktionsargument. Um z. B. die durch das Programm „`program.txt`“ definierte, 3-stellige Funktion an der Stelle (1, 5, 7) auszuwerten, sind die Parameter „`-e program.txt 1 5 7`“ anzugeben.
- t FILE, --test=FILE** Testet die durch das Programm definierte Funktion mit Hilfe einer Testdatei („FILE“). Eine Testdatei enthält Zeilen von Eingabetupeln und erwartetem Ausgabewert. Auf diese Weise können schnell und einfach viele Testfälle durchgeprüft werden. Eine Zeile der Form „1 5 7 123“ besagt dabei, dass die definierte, 3-stellige Funktion an der Stelle (1, 5, 7) den Wert 123 haben soll.
- T** Aktiviert die print-Ausgabe auch bei Tests. Im Normalfall werden bei der Ausführung von Tests mit der Option „-t“ die print-Ausgaben des While- bzw. Loop-Programms deaktiviert, um die Ausgabe übersichtlich zu halten. Mit dieser Option kann die Ausgabe der print-Anweisungen im Programm wieder aktiviert werden.

2.3 Hinweise

Aufgrund fundamentaler Probleme, aber auch bedingt durch Limitierungen der Sprache Python, kann der Syntaxprüfer bei Auswertung von Programmen nicht mit Sicherheit auf Nichtdefiniertheit prüfen. Im Allgemeinen gilt: wenn der Syntaxprüfer bei Auswertung mit „-e“ oder Tests mit „-t“ einen Rückgabewert des Programms meldet, so ist dieser korrekt. Es kann jedoch im Allgemeinen nicht von der Ausgabe „wahrscheinlich nicht definiert“ auf echte Nichtdefiniertheit geschlossen werden,² ebenso wenig kann von einer scheinbar unendlichen Ausführung (Endlosschleife) auf echte Nichtdefiniertheit geschlossen werden.³

3 Beispiel

Im Folgenden wird beschrieben, wie ein Programm erstellt und getestet werden kann, das das Quadrat der Eingabe berechnet. Genaugenommen soll das Programm die folgende 1-stellige Funktion berechnen:

$$\text{square}(n) = \begin{cases} n^2 & \text{falls } n > 0, \\ 0 & \text{sonst.} \end{cases}$$

Die Beschränkung auf positive Zahlen wird dabei die Implementierung ein wenig vereinfachen.

²Beispielsweise wird der Python-Interpreter ein tief rekursives Programm abbrechen, auch wenn es nach einigen weiteren Rekursionen einen definierten Wert annehmen würde. Die Begrenzung liegt hier darin, dass Python nur eine endliche Anzahl von geschachtelten Funktionsaufrufen erlaubt, um Endlosschleifen abfangen und behandeln zu können.

³Dies ist die Essenz des sogenannten Halteproblems: es ist für einen Algorithmus im Allgemeinen nicht möglich zu entscheiden, ob ein gegebener Algorithmus auf einer gegebenen Eingabe zu einem Ergebnis kommt, also anhält.

Python installieren Wenn das verwendete Betriebssystem eine Paketverwaltung bereitstellt (dies ist der Fall bei den meisten Linux-Varianten), so bietet es sich an, damit Python 3.1 oder neuer zu installieren.⁴ Ist dies nicht möglich, so kann Python unter <http://python.org/download/> für das jeweilige Betriebssystem heruntergeladen werden. Im Folgenden werden wir annehmen, dass Python im Ausführungspfad installiert wurde, d. h., dass Python unter der Kommandozeile direkt mit dem Befehl „python“ aufgerufen werden kann.

Syntaxprüfer entpacken Der Syntaxprüfer wird in einem zip-Archiv namens „syntax-checker.zip“ angeboten.⁵ Der Archivinhalt ist in ein Verzeichnis mit dem Namen „syntax-checker“ zu entpacken. Es bietet sich an, dieses Verzeichnis an einem Ort zu platzieren, den man leicht auf der Kommandozeile erreichen kann. Im Folgenden werden wir annehmen, dass zur Arbeit mit dem Syntaxprüfer ein (anfangs leeres) Verzeichnis namens „theoinf“ angelegt wurde, in das wir im Unterverzeichnis „syntax-checker“ den Syntaxprüfer entpackt haben. In „theoinf“ werden wir auch unsere Programmdateien ablegen.

Programm schreiben Es ist an der Zeit, das eigentliche Programm zu schreiben, welches die oben beschriebene Funktion *square* berechnen wird. Dies kann in einem beliebigen Texteditor geschehen. Wir werden das Programm in einer Datei „tutorial.py“ im Verzeichnis „theoinf“ ablegen. Das folgende Programm erfüllt die gestellten Anforderungen:

tutorial.py

```
1 def mul(a, b):
2     res = 0
3     for i in range(0, a):
4         res = (res + b)
5     return res
6
7 def square(n):
8     return mul(n, n)
```

Da in While- und Loop-Programmen keine Multiplikation definiert ist, mussten wir unsere eigene Implementierung programmieren. Es ist festzuhalten, dass die Funktion „mul“ nur für Aufrufe mit $a \geq 0$ das korrekte Ergebnis der Multiplikation liefert, in allen anderen Fällen wird 0 zurückgegeben. Dies trifft sich jedoch mit der Definition von *square*, weshalb wir in „square“ ohne weitere Fallunterscheidung *mul(n, n)* zurückgeben können.

Syntaxprüfer aufrufen In einer Konsole (auch: auf einem Terminal, auf der Kommandozeile) können wir nun den Syntaxprüfer auf unser Programm „tutorial.py“ anwenden. Der folgende Aufruf liefert uns das erste Ergebnis:

```
goll@thomson:~/theoinf$ python syntax-checker tutorial.py
[+] <WHILE>  tutorial.py is a while program.

[-] <LOOP>    tutorial.py is not a loop program because of the following:
             Function 'mul' lacks initialization of variables. (line 1, column 5)

             1.1:  def mul(a, b):
                     ^
```

Der Ausgabe zufolge handelt es sich also bei unserem Programm um ein While-Programm, d. h., wir haben keine verbotenen (aber im „normalen“ Python definierten) Sprachkonstrukte verwendet. Allerdings gibt der Syntaxprüfer auch aus, dass wir kein Loop-Programm vorliegen haben: es fehle die Variableninitialisierung in der Funktion „mul“ (und auch in der Funktion „square“).

⁴In Debian und Ubuntu ist dies beispielsweise das Paket „python3“ oder alternativ „python3.1“. Pakete ohne expliziten Versionszusatz beziehen sich derzeit meist noch auf das ältere Python 2.x.

⁵Alternativ auch als gzip-komprimierter *tarball* unter dem Namen „syntax-checker.tar.gz“.

Nachbesserung Wir wollen aus unserem While-Programm durch geringfügige Anpassung ein Loop-Programm machen. Dies hat den Vorteil, dass das Programm garantiert terminiert, d. h., auf jede Eingabe wird unser Programm dann eine definierte und eindeutige Ausgabe liefern. Dies ist zwar jetzt bereits der Fall, kann aber dem Programm rein syntaktisch noch nicht angesehen werden. Wir fügen die geforderten Variableninitialisierungen hinzu und korrigieren „tutorial.py“ somit wie folgt:

tutorial.py

```
1 def mul(a, b):
2     [res, i] = [0, 0]
3     for i in range(0, a):
4         res = (res + b)
5     return res
6
7 def square(n):
8     [] = []
9     return mul(n, n)
```

Wenn wir nun erneut den Syntaxprüfer ausführen, erhalten wir die gewünschte Ausgabe:

```
goll@thomson:~/theoinf$ python syntax-checker tutorial.py
[+] <LOOP>    tutorial.py is a loop program (and thus while program).
```

Programm testen Bisher haben wir vom Syntaxprüfer nur die rein syntaktische Aufgabe abverlangt, eine Eingabe auf While- bzw. Loop-Konformität zu prüfen. Mit den Parametern „-e“ und „-t“ können wir die definierte Funktion *square* nun aber auch ganz konkret auf bestimmte Eingaben auswerten. Der folgende Aufruf berechnet z. B. das Quadrat von 17:

```
goll@thomson:~/theoinf$ python syntax-checker -e tutorial.py 17
[+] <LOOP>    tutorial.py is a loop program (and thus while program).
```

I will now try and evaluate the function defined by tutorial.py.

```
---[ start of print output ]-----
---[ end of print output ]-----
```

The result is:
square(17) = 289

Das Programm liefert das erwartete Ergebnis 289 ($= 17^2$). Bei der Auswertung werden zwischen den Zeilen „start of print output“ und „end of print output“ die Argumente von vorhandenen print-Anweisungen ausgegeben. Dieses Hilfskonstrukt, welches die Berechenbarkeit nicht beeinflusst, erlaubt eine elementare Fehlersuche.

Darüber hinaus können wir von einer weiteren Fähigkeit des Syntaxprüfers Gebrauch machen: wenn wir beispielsweise eine Datei „test.txt“ mit folgendem Inhalt anlegen, können wir Tests auch automatisieren:

test.txt

```
1 1 1
2 2 4
3 3 9
4 4 16
5 123 15129
6 999 0
7 0 0
8 -1 0
9 -123 0
```

Diese Datei enthält Eingabe- und erwartete Ausgabewerte der zu testenden Funktion *square*, und zu Demonstrationszwecken einen offenbar falschen Test in der Zeile „999 0“. Ein Aufruf mit dem Parameter „-t“ testet nun alle hinterlegten Werte und gibt zu jedem Test aus, ob dieser erfolgreich war:

```
goll@thomson:~/theoinf$ python syntax-checker -t test.txt tutorial.py
[+] <LOOP>    tutorial.py is a loop program (and thus while program).
```

I will now try and test tutorial.py according to test.txt.

The test results are:

```
[OK] square(1) = 1
[OK] square(2) = 4
[OK] square(3) = 9
[OK] square(4) = 16
[OK] square(123) = 15129
[ ] square(999) = 998001, but expected 0
[OK] square(0) = 0
[OK] square(-1) = 0
[OK] square(-123) = 0
```

```
Out of 9 tests, [OK] 8 passed and [ ] 1 failed.
```

Die Ausgabe gibt Aufschluss darüber, welche Tests fehlgeschlagen sind: in unserem Falle ist dies nur unser bewusst ungültiger Test. Davon abgesehen funktioniert unser Programm korrekt und berechnet (zumindest für diese Beispielwerte) tatsächlich die Quadratzahlen für positive Eingaben und gibt ansonsten den Wert 0 zurück.

3.1 Im CIP-Pool

Im Folgenden wird kurz umrissen, wie die obige Schritt-für-Schritt-Anleitung im CIP-Pool der Mathematik und Informatik in Würzburg umgesetzt werden kann. Dabei sind die genannten Befehle auf der Kommandozeile bzw. in einem Terminal (xterm o. ä.) zu starten.

Python installieren Dies ist nicht nötig, da Python auf den Rechnern des CIP-Pools bereits vorinstalliert ist. Der Befehl „python“ startet derzeit Python 2.6.6, der Befehl „python3“ startet Python 3.1.2. Je nach Komplexität der Programme, die wir schreiben, und welche Sprachfeatures sie verwenden, ist Python 3.x erforderlich. Wir werden daher im Folgenden grundsätzlich „python3“ zum Aufruf verwenden.

Verzeichnis anlegen Für die folgenden Befehle werden wir uns ein eigenes Verzeichnis zur Vorlesung im Heimatverzeichnis anlegen. Dort werden wir den Syntaxprüfer sowie unsere eigenen Programme ablegen. Das Verzeichnis müssen wir selbstverständlich nur einmal erstellen. Dies erreichen wir wie folgt:

```
s123456@aurich:~$ mkdir theoinf
```

In Verzeichnis wechseln Damit die folgenden Anweisungen tatsächlich im neuen Verzeichnis ausgeführt werden, wechseln wir in dieses:

```
s123456@aurich:~$ cd theoinf
```

Syntaxprüfer herunterladen Da man sich zum Herunterladen des Syntaxprüfers bei WueCampus anmelden muss, ist dies schneller in einem graphischen Browser erledigt. Beispielsweise mit Iceweasel laden wir also den Syntaxprüfer herunter (im tar.gz-Format) und speichern die Datei im neuen Verzeichnis „theoinf“ im Heimatverzeichnis.

Syntaxprüfer entpacken Nun entpacken wir das Archiv, das den Syntaxprüfer enthält:

```
s123456@aurich:~/theoinf$ tar xzf syntax-checker.tar.gz
```

Dadurch sollte ein neuer Ordner namens „syntax-checker“ unter „theoinf“ hinzukommen.

Programm schreiben Unsere While- und Loop-Programme schreiben wir in einem Texteditor unserer Wahl und platzieren sie im „**theoinf**“-Verzeichnis. Wir nehmen an, wir hätten das Programm „**tutorial.py**“ auf diese Weise reproduziert und in diesem Verzeichnis abgelegt.

Syntaxprüfer aufrufen Den Syntaxprüfer rufen wir aus dem Verzeichnis „**theoinf**“ heraus wie folgt auf und wenden ihn auf die dort hinterlegte Datei „**tutorial.py**“ an:

```
s123456@aurich:~/theoinf$ python3 syntax-checker tutorial.py
[+] <WHILE>  tutorial.py is a while program.

[-] <LOOP>   tutorial.py is not a loop program because of the following:
            Function 'mul' lacks initialization of variables. (line 1, column 5)

            1.1:  def mul(a, b):
                    ^
```

Die anderen, weiter oben beschriebenen Syntaxprüfer-Befehle lassen sich auf die gleiche Weise ausführen. Parallel dazu kann natürlich im Texteditor die Datei „**tutorial.py**“ weiter bearbeitet und im offenen Terminal immer wieder getestet werden.